

Projectverslag Computer Graphics 2025-2026

Janssen Maximiliaan (2467787)

Neys Michael (2467626)

24 mei 2026

1 Inleiding

Dit verslag beschrijft de implementatie van het Computer Graphics project voor het academiejaar 2025-2026. In onze applicatie hebben we een 3D-omgeving gebouwd met OpenGL (via GLFW en GLAD) waarin een bij (bee) over een traject van Bézier-curves vliegt. Dit verslag geeft een overzicht van de geïmplementeerde functionaliteiten en de gevolgde planning.

2 Geïmplementeerde Functionaliteiten

2.1 Curve en Animatie

Het traject van de bij is opgebouwd uit Bézier-curves aan de hand van 45 hardcoded control points in de `BezierPath` klasse. Om ervoor te zorgen dat de bij met een constante snelheid beweegt, en niet afhankelijk is van de parameter t , is een evaluatie op basis van booglengte geïmplementeerd. Tijdens de initialisatie wordt in de functie `buildLUT` een Look-Up Table opgesteld die de relatie tussen de parameter t en de afgelegde afstand bijhoudt.

2.2 Modellen, Textures en Visualisatie

Voor de omgeving hebben we gebruikgemaakt van een procedureel gegenereerd voxel-achtig terrein in de `Terrain` klasse. Er is een station gedefinieerd, omringd door heuvelachtig terrein. We maken gebruik van diffuse textures en fallback kleuren voor de modellen.

2.3 Camera's

Er zijn twee camera-modi geïmplementeerd die gewisseld kunnen worden met de ‘C’-toets:

- Een free-look/fly camera waarmee de hele scène vrij verkend kan worden.
 - W, A, S, D: Standaard navigatie (vooruit, links, achteruit, rechts).
 - Spatiebalk: Verticaal omhoog bewegen.
 - Control (Ctrl): Verticaal omlaag bewegen.
 - Shift: Versnellen/sprinten in de richting waarin je momenteel beweegt.
- Een first-person camera die vastgemaakt is aan de bij, waarbij de View-matrix wordt berekend aan de hand van de actuele positie en kijkrichting van de bij op de curve (`glm::lookAt`).

2.4 Belichting

De scène maakt gebruik van per-pixel belichting in de `lighting.frag` shader. Er is een globaal “maanlicht” (basislicht) en er zijn 18 lantaarns (point lights) door de scène verspreid. De invloed

van deze lokale lichtbronnen neemt realistisch af over afstand door middel van attenuatie (constante, lineaire en kwadratische afzwakking). Met de **‘R’-toets** kunnen de Redstone lampen getoggled worden.

2.5 Convolutie en Post-processing (Bloom)

We hebben een post-processing pipeline opgezet door de scène te renderen naar een Framebuffer Object (FBO).

- **Convolutie:** Via de toetsen **1, 2 en 3** kan geschakeld worden tussen geen effect, een Gaussian Blur en Edge Detection (Laplaciaan/Sobel).
- **Bloom:** Om de lantaarns een neon/halo effect te geven, wordt eerst een brightness-extractie uitgevoerd (`bloom_bright.frag`). Vervolgens wordt dit resultaat via ‘ping-pong’ framebuffers in meerdere passes (15 keer) geblurred om een zachte gloed te creëren. Tot slot wordt dit gecombineerd met de originele render. Het effect is in- en uit te schakelen met de **‘B’-toets**.

2.6 Interactie

Interactie in de applicatie is toegevoegd aan de hand van **raycasting**. Wanneer de gebruiker de linkermuisknop indrukt, wordt er vanuit het midden van het scherm (vergelijkbaar met een crosshair in een first-person shooter) een straal (ray) in de 3D-wereld geschoten.

De implementatie hiervan is als volgt opgebouwd:

- **Picking FBO & Shader:** Er wordt gebruikgemaakt van een apart Framebuffer Object (`pickingFBO`) en een specifieke shader (`pickingShader`).
- **Off-screen rendering:** Zodra de gebruiker klikt, worden de interacteerbare objecten (in dit geval de Redstone lampen) naar deze onzichtbare buffer getekend met een unieke, egale kleur (puur rood: RGB 255, 0, 0). De belichting en textures worden hierbij overgeslagen.
- **Pixel uitlezen:** Vervolgens wordt met de functie `glReadPixels` de exacte kleurwaarde van de pixel in het midden van het scherm (`screenWidth / 2, screenHeight / 2`) uitgelezen.
- **Resultaat:** Als de gelezen kleur overeenkomt met de ingestelde rode kleur, detecteert het programma een ‘hit’. De status van de lampen (`redstoneLampsOn`) wordt dan getoggled om ze allemaal tegelijk in of uit te schakelen (texture aan te passen) en de lichtbronnen buiten het station uit te schakelen.

2.7 Chroma-keying

Voor de implementatie van chroma-keying werd een textured quad toegevoegd aan de scène via de klasse `BluePlane`. Deze quad bestaat uit twee driehoeken waarop een overlay-afbeelding wordt geprojecteerd met behulp van texture coordinates.

De overlay-textuur wordt ingeladen via `stb_image` en doorgestuurd naar een aparte fragment shader (`bluePlane.frag`). In deze shader wordt chroma-keying toegepast door een vooraf bepaalde achtergrondkleur (blue-screen) te detecteren en transparant te maken via `discard`. Hierdoor blijven enkel de gewenste delen van de afbeelding zichtbaar bovenop de gerenderde 3D-scène.

Om de afbeelding correct weer te geven zonder vervorming wordt rekening gehouden met de aspect ratio van de originele textuur. De afmetingen van het vlak worden dynamisch aangepast op basis van de breedte- en hoogteverhouding van de geladen afbeelding.

Tijdens het renderen wordt backface culling tijdelijk uitgeschakeld zodat de overlay vanuit beide richtingen correct zichtbaar blijft. De overlay kan bovendien dynamisch geactiveerd of verborgen worden via een boolean uniform in de shader.

3 Niet-geïmplementeerde Functionaliteiten

Alle basisfunctionaliteiten uit de opgave werden geïmplementeerd.

4 Planning en Werkverdeling

We hebben beide actief meegewerkt aan het project via Git. In de onderstaande tabel wordt de tijdsbesteding (per persoon, per dag/periode en het aantal uren) inzichtelijk gemaakt aan de hand van onze bijgehouden urenregistratie, zoals vereist in de opgave.

Datum	Tijd	Persoon	Taak / Beschrijving
10 april	2 min	Michael	Werkuren sheet maken
10 april	10 min	Maximiliaan	Trello opbouwen
14 - 15 april	10 uur	Maximiliaan	Base template opzetten
14 april	1.5 uur	Michael	Clean up repo & GitHub workflows (Linux/Windows)
19 april	3.5 uur	Michael	Bezier curve wiskunde & display implementatie
23 - 30 april	11 uur	Maximiliaan	Texture displaying
3 - 5 mei	15 uur	Michael	Track door treinstation modelleren/plaatsen
5 mei	1.5 uur	Michael	Bij de track laten volgen (constante snelheid & kijkrichting)
5 - 23 mei	5.5 uur	Michael	Belichting van de lantaarns
6 - 14 mei	7 uur	Maximiliaan	Terrain toevoegen
6 - 14 mei	5 uur	Maximiliaan	Skybox toevoegen
10 mei	1.5 uur	Samen	Call voor overleg
14 mei	1 uur	Maximiliaan	Meer controls voor free-camera
17 - 23 mei	15 uur	Maximiliaan	Convolutie algemeen opzetten & toevoegen
17 - 23 mei	3 uur	Maximiliaan	Gaussian Blur
17 - 23 mei	5 uur	Maximiliaan	Laplaciaan kernel
23 - 24 mei	7 uur	Maximiliaan	Bloom implementatie
23 - 24 mei	3 uur	Maximiliaan	First-person camera bij
23 - 24 mei	2 uur	Samen	Verslag schrijven
24 mei	2.5 uur	Michael	Picking voor interactie (raycasting)
24 mei	30 min	Michael	Resizing van windows correct afhandelen
24 mei	15 min	Michael	Cursor popout (mouse capture met ESC)
24 mei	3 uur	Michael	Opschonen van de code
24 mei	1 uur	Maximiliaan	Chroma-keying
24 mei	1 uur	Michael	Windows build werkende krijgen

Tabel 1: Overzicht van de daadwerkelijk gevolgde planning, taakverdeling en bestede tijd.